

Tarea 3: Simulador Gráfico en C++ y Qt de EloTelTags y Aplicación Find My

Lea detenidamente la tarea. **Si algo no lo entiende, consulte.** Si es preciso, se incorporarán aclaraciones al final.

1. Objetivos de la tarea

- Modelar objetos reales como objetos de software.
- Ejercitar la creación y extensión de clases dadas para satisfacer nuevos requerimientos.
- Reconocer clases y relaciones entre ellas en códigos fuentes C++.
- Ejercitar la compilación y ejecución de programas C++ desde una consola de comandos.
- Ejercitar la configuración de un ambiente de trabajo para desarrollar aplicaciones en C++ y Qt.
- Manejar proyectos de software vía Git.
- Ejercitar la preparación y entrega de resultados de software (readme, documentación).
- Familiarización con desarrollos "iterativos" e "incrementales".
- Desarrollar software con interfaz gráfica utilizando la biblioteca Qt.

2. Descripción General

Esta tarea busca programar, usando C++ y Qt, las funcionalidades de la Tarea 2 ya desarrollada en Java y JavaFX para simular la interacción entre dispositivos de rastreo EloTelTags, celulares y tablets. La descripción del funcionamiento de EloTelTags, celulares y tables es la misma de la Tarea 2, revise allí esos detalles.

3. Funcionamiento de la Aplicación

En esta oportunidad se espera que su programa se ejecute desde un IDE (Qt Creator de preferencia o alternativamente Visual Studio Code).

Al igual que la Tarea 2, las últimas etapas de esta tarea usan el paradigma “programación conducida por eventos”. Los eventos del usuario y los del programa gobernarán la ejecución. Las primeras etapas está orientadas a ejercitar la programación en C++ con la lectura de archivos de entrada, la creación de objetos y la simulación simple sin considerar tiempo real.

Como en la Tarea 2, la configuración de los dispositivos es tomada de un archivo (por ejemplo config.txt) el cual es especificado a través de una instancia de [QFileDialog](#). El formato para ese archivo es:

```
<archivo_imagen_de_fondo>
<delta_tiempo>
<número de personas>
<nombre 1º persona> <número de EloTelTags> (0 | 1)
<celular_x> <celular_y> <rapidez> <ángulo> <delta_ángulo>
(<nombre EloTelTag> <tag_x> <tag_y> <rapidez> <ángulo> <delta_ángulo>) *
[<tablet_x> <tablet_y> <rapidez> <ángulo> <delta_ángulo>]
```

La primera línea contiene la imagen a utilizar como fondo para el territorio donde se mueven los equipos. Para su conveniencia, puede tomar [ésta](#).

delta_tiempo corresponde al valor discreto en segundos en que avanza el tiempo de simulación, por ejemplo, para mover los equipos (tags, celulares, tablets).

Para cada equipo, las coordenadas corresponden a su posición inicial (en unidad de pixeles), la rapidez es la magnitud de la velocidad (en pixeles/s) con que se mueve por el territorio, el ángulo (en grados) es el inicial de su velocidad, y delta_ángulo (en grados) define el rango (-delta_ángulo, +delta_ángulo) de donde aleatoriamente y a intervalos regulares delta_tiempo se modifica el ángulo de la velocidad con que el equipo se desplaza levemente.

Un ejemplo para este archivo es:
Placeres.jpg

0.1
3
Pedro 2 1
1000 100 20 30 8
maleta 50 170 15 -20 10 mochila 520 130 10 -130 30
305 99 15 150 20
Juan 1 0
500 260 15 -120 20
llaves 450 500 10 -90 10
Diego 2 1
335 355 10 -20 20
llaves 400 100 40 90 30 mochila 405 431 15 -10 20
210 310 15 180 20

Al inicio del programa, se muestra una interfaz como la Figura 1.

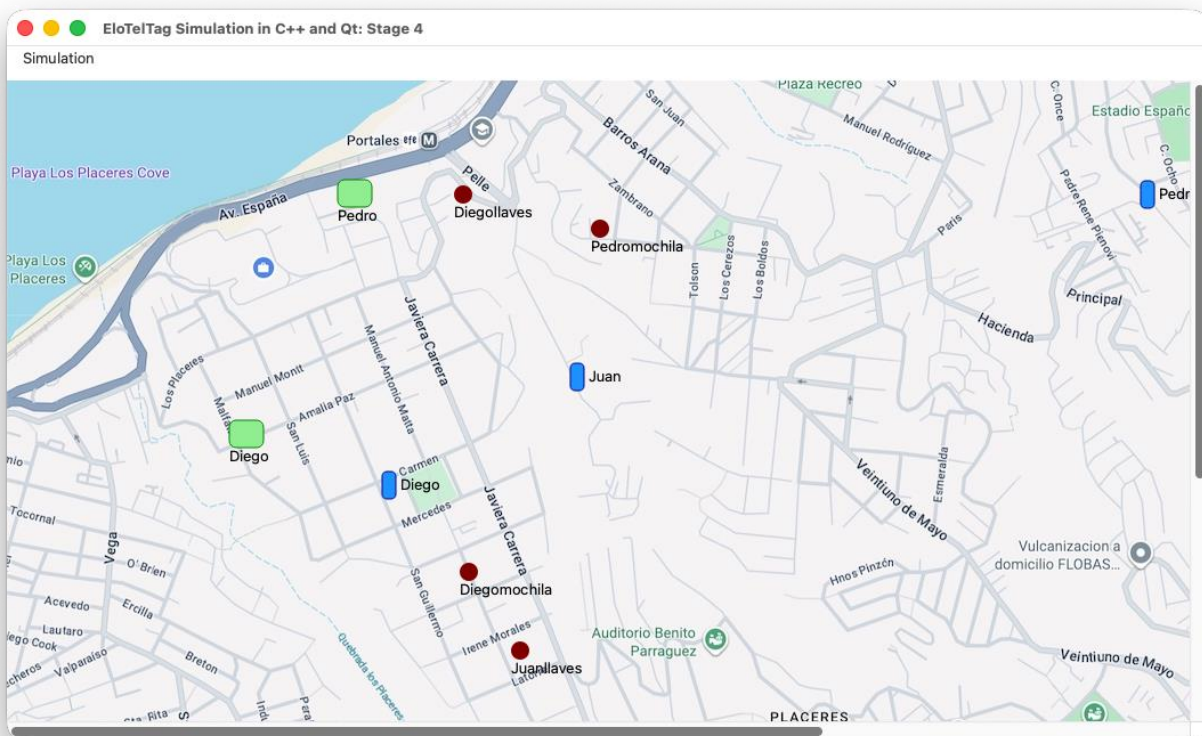


Figura 1: Ejemplo de interfaz con sólo tres celulares
El menú Simulation ofrece dos opciones: "Play" y "Pause" al estilo de la Figura 2.



Figura 2: Opciones del menú Simulation

Al seleccionar "Play" los equipos se mueven por el territorio manteniéndose confinados a él. Al llegar al borde del Territorio, el cual coincide con el tamaño de la imagen usada, los equipos rebotan en el borde. Al rebotar mantienen la componente de la velocidad en sentido paralelo al borde e invierten la componente de la velocidad perpendicular al borde. Durante la simulación, celulares reportan su posición cada 4 segundos, los EloTelTags y las tablets buscan celulares cercanos cada t_{bip} segundos sin mantener una sincronización de fase (no lo hacen al mismo tiempo). Este tiempo es de 4 segundos para todos los Tags y de 5 segundos para todos los tablets. En la interfaz, la búsqueda de celulares se manifiesta con la aparición de un círculo que crece desde el equipo hasta llegar a un radio de 50 pixeles en 1 segundo. Cuando un celular es alcanzado

por la señal de un tag o tablet, como la Tarea 2, el celular reporta a la nube el nombre del dispositivo, su dueño/a y la posición del celular en ese momento.

La función FindMy de tablets y celulares se activa presionando en el interior de la vista del equipo. Se muestra así un menú emergente/flotante con una única opción como se muestra en Figura 3. Al seleccionar la opción "Find My" se muestra una nueva ventana con la misma información descrita para la Tarea 2.

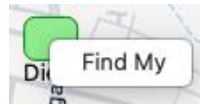


Figura 3: Menú emergente al presionar en celular o tablet

Mientras se mantiene activa la ventana FindMy, ésta se actualiza cada un segundo con la información disponible en la nube según el movimiento de los equipos.

Notar que en esta tarea tags y tables no emiten sonido al activar su búsqueda de celulares; sin embargo, se agrega la función traza. Asociado a cada equipo mostrado en la ventana "Find My", se incluye un botón que al presionarlo mostrará en el mapa una línea con la trayectoria seguida por se equipo desde ese momento. Al presionar nuevamente el mismo botón, la trayectoria desaparece.

4. Desarrollo en Etapas

Para llegar al resultado final de esta tarea usted avanzará en pasos "[Iterativos y crecientes \(o incrementales\)](#)" comunes en metodologías de desarrollo de software actuales. Usted y su equipo irán desarrollando etapas donde los requerimientos del sistema final son abordados gradualmente. En cada etapa usted y su equipo obtendrá una solución que funciona para un subconjunto de requerimientos finales o bien es un avance hacia ellos. En AULA se dispondrá el recurso para subir un archivo de texto con el enlace al repositorio de GitHub de su grupo. **Su equipo deberá generar una solución para cada una de las etapas** aun cuando la última integre las primeras. **Los archivos readme.txt y de documentación deben ser preparados sólo para la última etapa. Prepare en su repositorio Git un proyecto IntelliJ o Visual Studio Code para cada una de las etapas.** Esto tiene por objetivo conocer la metodología de desarrollo iterativo e incremental.

4.1. Etapa 1: Lectura de configuración e imprima posición de celulares en salida estándar

Desarrolle las clases Cellular, CellularView, Equipo, Territory y TerritoryView. Cree el programa stage1 el cual usa un método estático de QFileDialog para seleccionar el archivo de configuración, el cual lee y muestra una ventana similar a la Figura 1 pero celulares únicamente, y sin menú.

4.2. Etapa 2: Lectura de configuración completo y despliegue del movimiento de equipos

Agregue las clases CellularView, TerritoryView, EloTelTag, EloTelTagView, Tablet, TabletView, ETNube y otras que estime. Su programa stage2 lee el archivo de configuración y muestra una ventana como la Figura 1 con todos los equipos configurados. Los equipos aparecen moviéndose hasta terminar del programa vía el cierre de su ventana.

4.3. Etapa 3: Reporte de posiciones hacia ETNube

Agregue la clase ETNube. En esta etapa los celulares deben reportar a la nube las posiciones de los equipos rastreables cercanos y un menú emergente debe mostrar la posición de los equipos según la opción FindMy. Tags y Tablets muestran su señal de radar. La ventana con la información de los equipos de esa persona no se actualiza.

4.4. Etapa 4: Cumplimiento de todas las funcionalidades

Complete lo necesario para cumplir con las funcionalidades descritas en el punto 3.

4.5. Extra-crédito: Si desarrolla esta parte, lo debe indicar en su readme. Su equipo puede recibir hasta 10 puntos adicionales si el menú emergente incluye la opción GFindMy, por Graphics Find My. Al seleccionar esta opción se muestra una ventana con la misma imagen de la ventana principal de la

simulación y vistas de los equipos de la persona. Las vistas de los equipos se deben actualizar cada un segundo para reflejar la última información disponible en la nube.

5. Elementos por considerar en su entrega

En AULA habrá un buzón para subir únicamente el enlace al repositorio de GitHub. En dicho repositorio deberá incluir el readme y la documentación, indicando todas las etapas desarrolladas y la información necesaria para revisar y ejecutar el proyecto. **Su equipo deberá entregar una solución para cada una de las etapas** aun cuando la última integre las primeras. **El readme y archivo de documentación deben entregarse sólo para la última etapa desarrollada.**

En su archivo de documentación (pdf o html) incorpore el diagrama de clases de su aplicación (última etapa desarrollada)

Revise lo indicado en “[Normas de Entrega de Tareas](#)”, en caso de diferencias, lo señalado en este escrito tiene precedencia.